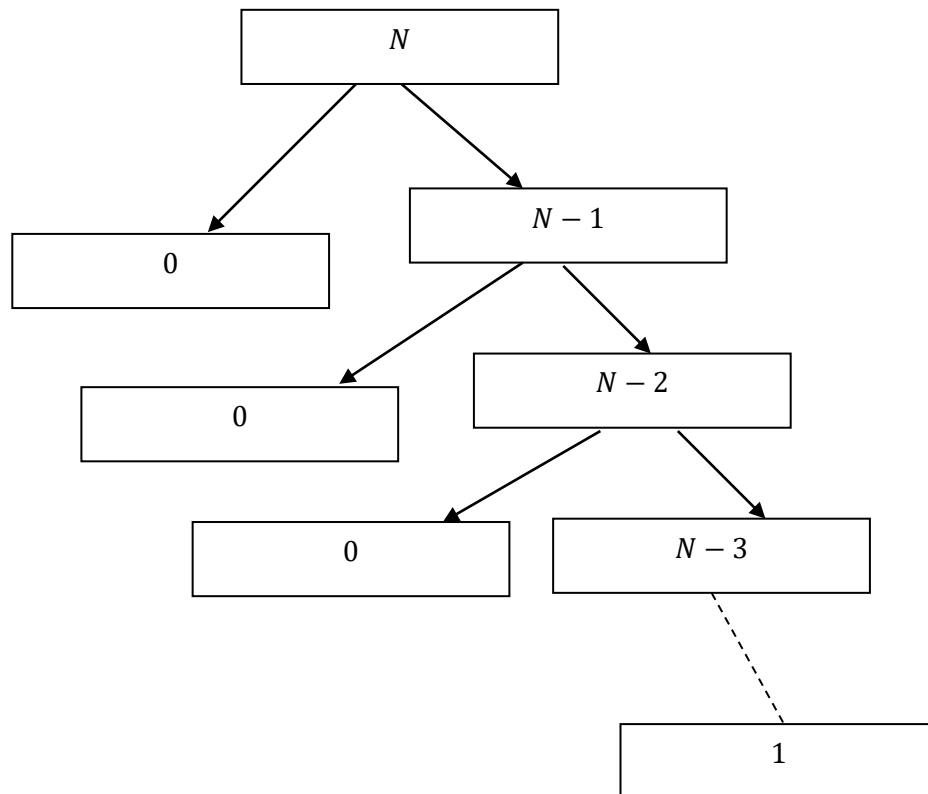


Multi – level Linked List

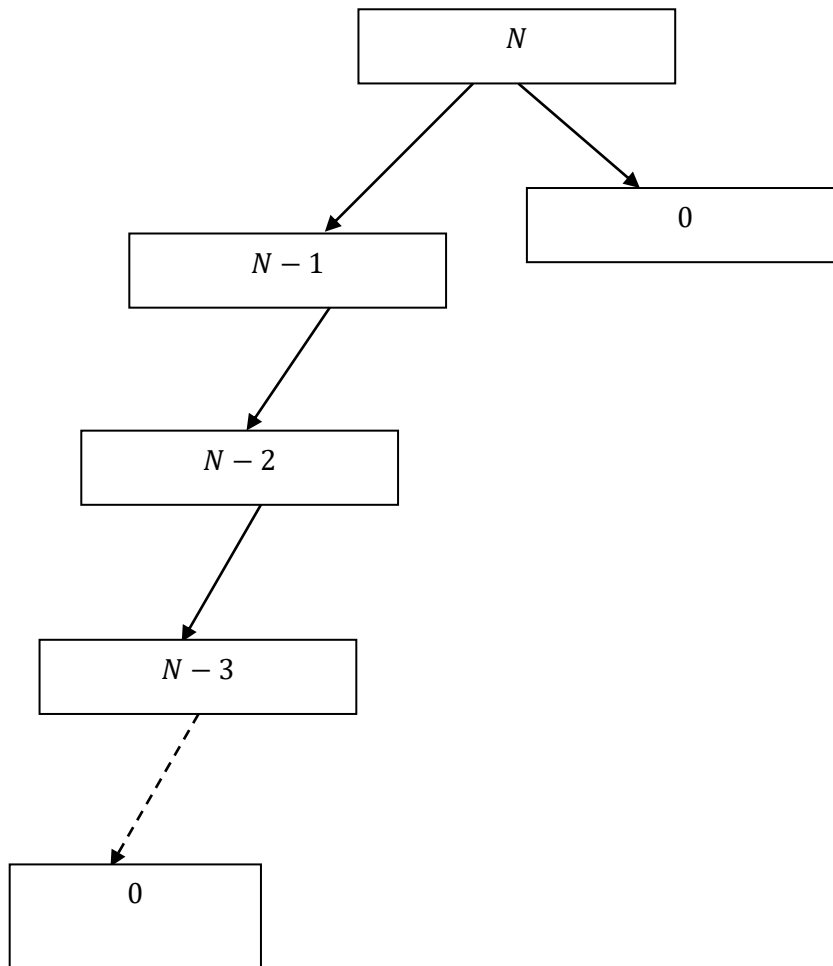
–Modified Insert After Element

–Time Complexity Analysis – Worst Case[Right Skewed and Left Skewed]

Right Skewed



Left Skewed



For both the cases we have :

1) Right – Skewed ($k = 0$) : $T(N) = T(N - 1) + c$

2) left – Skewed ($k = N - 1$) : $T(N) = T(N - 1) + c$

And $T(0) = c$ or say 1 unit of time (considered).

A) By Substitution Method

$$T(N) = T(N - 1) + c$$

$$\therefore T(N - 1) = T(N - 2) + c$$

Substituting in $T(N) = T(N - 1) + c$, we get:

$$\Rightarrow T(N) = T(N - 2) + c + c \Rightarrow T(N - 2) + 2c$$

$$\therefore T(N - 2) = T(N - 3) + c$$

Substituting in $T(N) = T(N - 2) + 2c$, we get:

$$\Rightarrow T(N) = T(N - 3) + 2c + c \Rightarrow T(N - 3) + 3c$$

if it goes for `i` time , we get:

$$\Rightarrow T(N) = T(N - i) + ic$$

As , we see $T(0) = 1$, i. e. Base Case when $T(0)$.

$$\therefore N - i = 0 \text{ [0 for } T(0)]$$

$$\therefore i = N$$

$$\Rightarrow T(N) = T(N - N) + N \cdot c$$

$$\Rightarrow T(N) = T(0) + N \cdot c$$

As, $T(0)$ is constant, we can write it as: $T(0) = c_0$

$$\Rightarrow T(N) = c_0 + N \cdot c$$

By applying Big - O, removing all constants we get:

$$\Rightarrow T(N) = O(N)$$

B) Recurrence Tree Method

<i>Level (k)</i>	<i>No. of problems</i>	<i>Problem Size</i>	<i>Work /Cost Per Problem</i>	<i>Work done = Work per Problem × No. of Problems</i>
0	1	$n - 0 = n$	C	$1 \times C = C$
1	1	$n - 1$	C	$1 \times C = C$
2	1	$n - 2$	C	$1 \times C = C$
⋮	⋮	⋮	⋮	⋮
$n - 1$	1	$n - (n - 1)$ $=$ 1	C	$1 \times C = C$
n (Base Case)	1	$n - n = 0$ (Base Case)	$T(0)$	$1 \times T(0)$ $= T(0)$

Why level `n` is Base Case?

Ans : for , level `k` :, Base Case = $T(0) \therefore n - k = 0$

$\Rightarrow k = n$.

Total Cost = $C + C + C + \dots (N)\text{times} + T(0)$.

= $C(N) + T(0)$

We know , $T(0)$ is 1, we can go with 1 ,but let instead lets write $T(0) = C_0$,representing constant.

$$\textbf{\textit{Total Cost}} = C(N) + C_0$$

As, C and C_0 both are constant Applying Big O , we get:

$$= O(N)$$
